

DeepSeekMath-V2: Towards Self-Verifiable Mathematical Reasoning

DeepSeekMath-V2: 走向可自我验证的数学推理

Zhihong Shao, Yuxiang Luo, Chengda Lu†, Z.Z. Ren

邵智宏, 罗宇翔, 卢成达 †, Z.Z. 任

Jiewen Hu, Tian Ye, Zhibin Gou, Shirong Ma, Xiaokang Zhang

胡洁文, 田烨, 苟志斌, 马世荣, 张晓康

DeepSeek-AI

DeepSeek-AI

zhihongshao@deepseek.com

zhihongshao@deepseek.com

<https://github.com/deepseek-ai/DeepSeek-Math-V2>

<https://github.com/deepseek-ai/DeepSeek-Math-V2>

Abstract

摘要

Large language models have made significant progress in mathematical reasoning, which serves as an important testbed for AI and could impact scientific research if further advanced. By scaling reasoning with reinforcement learning that rewards correct final answers, LLMs have improved from poor performance to saturating quantitative reasoning competitions like AIME and HMMT in one year. However, this approach faces fundamental limitations. Pursuing higher final answer accuracy doesn't address a key issue: correct answers don't guarantee correct reasoning. Moreover, many mathematical tasks like theorem proving require rigorous step-by-step derivation rather than numerical answers, making final answer rewards inapplicable. To push the limits of deep reasoning, we believe it is necessary to verify the comprehensiveness and rigor of mathematical reasoning. Self-verification is particularly important for scaling test-time compute, especially for open problems without known solutions. Towards self-verifiable mathematical reasoning, we investigate how to train an accurate and faithful LLM-based verifier for theorem proving. We then train a proof generator using the verifier as the reward model, and incentivize the generator to identify and resolve as many issues as possible in their own proofs before finalizing them. To maintain the generation-verification gap as the generator becomes stronger, we propose to scale verification compute to automatically label new hard-to-verify proofs, creating training data to further improve the verifier. Our resulting model, DeepSeekMath-V2, demonstrates strong theorem-proving capabilities, achieving gold-level scores on IMO 2025 and CMO 2024 and a near-perfect 118/120 on Putnam 2024 with scaled test-time compute. While much work remains, these results suggest that self-verifiable mathematical reasoning is a feasible research direction that may help develop more capable mathematical AI systems.

大语言模型在数学推理方面取得了显著进展, 这为人工智能提供了一个重要的测试平台, 并且如果进一步发展, 可能会对科学研究产生影响。通过使用奖励正确最终答案的强化学习来扩展推理能力, LLMs 在过去一年中从表现不佳提升到了在 AIME 和 HMMT 等定量推理竞赛中达到饱和水平。然而, 这种方法存在根本性的局限性。追求更高的最终答案准确性并没有解决一个关键问题: 正确的答案并不保证正确的推理过程。此外, 许多数学任务, 如定理证明, 需要严谨的逐步推导, 而不是数值答案, 因此最终答案奖励并不适用。为了推动深度推理的极限, 我们认为有必要验证数学推理的全面性和严谨性。自我验证对于扩展测试时的计算尤为重要, 尤其是在没有已知解的开放性问题。为了实现可自我验证的数学推理, 我们研究如何训练一个准确且忠实的基于 LLM 的验证器用于定理证明。然后, 我们使用该验证器作为奖励模型来训练一个证明生成器, 并激励生成器在最终确定其证明之前尽可能识别和解决其中的问题。为了在生成

器变得更强时保持生成-验证之间的差距，我们提出扩展验证计算以自动标记新的难以验证的证明，从而创建训练数据以进一步提高验证器的性能。我们的结果模型 DeepSeekMath-V2 展示了强大的定理证明能力，在 IMO 2025 和 CMO 2024 上取得了金牌级别的成绩，并且在 Putnam 2024 上以扩展的测试时计算达到了接近完美的 118/120 分。尽管还有很多工作要做，这些结果表明，自我验证的数学推理是一个可行的研究方向，可能会帮助开发更强大的数学人工智能系统。

1. Introduction

1. 引言

The conventional approach to reinforcement learning (RL) for mathematical reasoning involves rewarding large language models (LLMs) based on whether their predicted final answers to quantitative reasoning problems match ground-truth answers (Guo et al., 2025). This methodology suffices to allow frontier LLMs to saturate mathematical competitions that primarily evaluate final answers, such as AIME and HMMT. However, this reward mechanism has two fundamental limitations. First, it serves as an unreliable proxy for reasoning correctness – a model can arrive at the correct answer through flawed logic or fortunate errors. Second, it is

传统的强化学习 (RL) 方法用于数学推理，通常通过奖励大型语言模型 (LLMs) 来实现，其依据是这些模型对定量推理问题的预测最终答案是否与真实答案匹配 (Guo 等, 2025)。这种方法足以让前沿的 LLMs 在主要评估最终答案的数学竞赛 (如 AIME 和 HMMT) 中达到饱和水平。然而，这种奖励机制存在两个根本性的局限性。首先，它作为推理正确性的不可靠代理——模型可以通过错误的逻辑或幸运的错误得出正确的答案。其次，它

inapplicable to theorem proving tasks, where problems may not require producing numerical final answers and rigorous derivation is the primary objective.

不适用于定理证明任务，因为在这些任务中，问题可能不需要产生数值性的最终答案，而严谨的推导才是主要目标。

Consequently, LLMs trained on quantitative reasoning problems with such final answer reward still frequently produce mathematically invalid or logically inconsistent natural-language proofs. Moreover, this training approach does not naturally develop the models' ability to verify proof validity – they exhibit high false-positive rates, often claiming incorrect proofs are valid even when they contain obvious logical flaws.

因此，即使在定量推理问题上使用最终答案奖励进行训练的大型语言模型 (LLMs)，仍然经常生成数学上无效或逻辑不一致的自然语言证明。此外，这种训练方法并未自然地培养模型验证证明有效性的能力——它们表现出较高的误报率，常常在证明中明显存在逻辑错误时，仍声称这些证明是有效的。

The lack of a generation-verification gap in natural-language theorem proving hinders further improvement. To address this, we propose developing proof verification capabilities in LLMs. Our approach is motivated by several key observations:

缺乏自然语言定理证明中的代际验证差距阻碍了进一步的改进。为了解决这个问题，我们提出在大型语言模型中开发证明验证能力。我们的方法受到以下几个关键观察的启发：

- Humans can identify issues in proofs even without reference solutions – a crucial ability when tackling open problems.
- A proof is more likely to be valid when no issues can be identified despite scaled verification efforts.
- The efforts required to identify valid issues can serve as a proxy for proof quality, which can be exploited to optimize proof generation.
- 人类可以在没有参考解的情况下识别证明中的问题——这对于解决开放性问题是一项关键能力。
- 当在扩大验证努力后仍无法识别任何问题时，证明更有可能是有效的。

- 识别有效问题所需的努力可以作为证明质量的代理指标，可用于优化证明生成。

We believe that LLMs can be trained to identify proof issues without reference solutions. Such a verifier would enable an iterative improvement cycle: (1) using verification feedback to optimize proof generation, (2) scaling verification compute to auto-label hard-to-verify new proofs, thereby creating the training data to improve the verifier itself, and (3) using this enhanced verifier to further optimize proof generation. Moreover, a reliable proof verifier enables us to teach proof generators to evaluate proofs as the verifier does. This allows a proof generator to iteratively refine its proofs until it can no longer identify or resolve any issues. In essence, we make the model explicitly aware of its reward function and enable it to maximize this reward through deliberate reasoning rather than blind trial-and-error.

我们认为，LLMs 可以被训练以在不参考解决方案的情况下识别证明中的问题。这样的验证器将能够实现一个迭代改进的循环：(1) 利用验证反馈来优化证明生成，(2) 扩展验证计算能力以自动标记难以验证的新证明，从而创建用于改进验证器本身的训练数据，(3) 使用这种增强的验证器进一步优化证明生成。此外，一个可靠的证明验证器使我们能够教导证明生成器以与验证器相同的方式评估证明。这使得证明生成器能够不断改进其证明，直到它无法再识别或解决任何问题。本质上，我们使模型明确地意识到其奖励函数，并使其能够通过有意识的推理而非盲目的试错来最大化这一奖励。

Built on DeepSeek-V3.2-Exp-Base (DeepSeek-AI, 2025), we developed DeepSeekMath-V2, a large language model optimized for natural-language theorem proving that demonstrates self-verifiable mathematical reasoning. Our model can assess and iteratively improve its own proofs, achieving gold-level performance in premier high-school mathematics competitions including IMO 2025 and CMO 2024. On the Putnam 2024 undergraduate competition, it scored 118/120, exceeding the highest score of 90¹ obtained by human participants.

基于 DeepSeek-V3.2-Exp-Base (DeepSeek-AI, 2025)，我们开发了 DeepSeekMath-V2，这是一个专为自然语言定理证明优化的大型语言模型，能够展示可自我验证的数学推理。我们的模型可以评估并迭代改进自身的证明，在包括 IMO 2025 和 CMO 2024 在内的顶级高中数学竞赛中取得了金牌级别的表现。在 Putnam 2024 本科生竞赛中，其得分达到 118/120，超过了由人类参赛者获得的最高分数 90¹。

2. Method

2. 方法

2.1. Proof Verification

2.1. 证明验证

2.1.1. Training a Verifier to Identify Issues and Score Proofs

2.1.1. 训练一个验证器以识别问题并评分证明

We developed high-level rubrics $\mathcal{J}_\nu$ for proof evaluation (see Appendix A.2) with the goal of training a verifier to evaluate proofs according to these rubrics, mirroring mathematical experts' assessment process. Specifically, given a problem X and a proof Y , the verifier $\pi_\varphi(\cdot|X, Y, \mathcal{J}_\nu)$ is designed to produce a proof analysis that first summarizes identified issues (if any) and then assigns a score based on three levels: 1 for complete and rigorous proofs with all logical steps clearly justified; 0.5 for proofs with sound overall logic but minor errors or omitted details; and 0 for fundamentally flawed proofs containing fatal logical errors or critical gaps.

我们开发了高水平的评分标准 $\mathcal{J}_\nu$ 用于证明评估（见附录 A.2），目的是训练一个验证器，使其能够根据这些评分标准评估证明，从而模拟数学专家的评估过程。具体而言，给定一个问题 X 和一个证明 Y ，验证器 $\pi_\varphi(\cdot|X, Y, \mathcal{J}_\nu)$ 被设计为生成一个证明分析，首先总结识别出的问题（如有），然后根据三个等级进行评分：1 分用于完全且严谨的证明，所有逻辑步骤都得到了明确的证明；0.5 分用于逻辑整体正确但有小错误或遗漏细节的证明；0 分用于根本性错误的证明，包含致命的逻辑错误或关键的缺失。

Curating Cold Start RL Data We constructed our initial training data through the following process:

Curating Cold Start RL Data 我们通过以下过程构建了初始训练数据:

1. We crawled problems from Art of Problem Solving (AoPS) contests ², prioritizing math olympiads, team selection tests, and post-2010 problems explicitly requiring proofs, totaling 17,503 problems. This problem set is denoted as $\mathcal{D}_p$.
2. We generated candidate proofs using a variant of DeepSeek-V3.2-Exp-Thinking. As this model was not optimized for theorem proving and tended to produce concise but error-prone outputs, we prompted it to iteratively refine its proofs over multiple rounds to improve comprehensiveness and rigor.
3. We randomly sampled proofs across diverse problem types (e.g., algebra and number theory) and had mathematical experts score each proof according to the evaluation rubrics described above.
4. 我们从 Art of Problem Solving (AoPS) 竞赛 ² 中爬取了问题, 优先选择数学奥林匹克、团队选拔考试以及明确要求证明的 2010 年后的问题, 共计 17,503 道题目。这个题目集被标记为 $\mathcal{D}_p$ 。
5. 我们使用 DeepSeek-V3.2-Exp-Thinking 的一种变体生成候选证明。由于该模型并未针对定理证明进行优化, 且倾向于生成简洁但容易出错的输出, 我们提示它在多轮中迭代地改进其证明, 以提高全面性和严谨性。
6. 我们在多种类型的问题 (例如代数和数论) 中随机采样证明, 并请数学专家根据上述描述的评估标准对每个证明进行评分。

This process yielded an initial RL dataset $\mathcal{D}_\nu = \{(X_i, Y_i, s_i)\}$, where each item consists of a problem X_i , a proof Y_i , and an overall proof score $s_i \in \{0, 0.5, 1\}$.

这个过程产生了一个初始的强化学习数据集 $\mathcal{D}_\nu = \{(X_i, Y_i, s_i)\}$, 其中每个条目包括一个问题 X_i 、一个证明 Y_i 以及一个总体证明评分 $s_i \in \{0, 0.5, 1\}$ 。

RL Objective. Building on a version of DeepSeek-V3.2-Exp-SFT which was supervised fine-tuned on reasoning data related to mathematics and code, we trained the model with reinforcement learning to produce proof analyses using two reward components:

RL 目标。基于一个名为 DeepSeek-V3.2-Exp-SFT 的版本, 该版本在与数学和代码相关的推理数据上进行了监督微调, 我们通过强化学习训练模型, 使用两个奖励组件来生成证明分析:

- Format reward R_{format} : An indicator function that enforces the model to generate both a summary of identified issues and a proof score, by checking whether the final response contains the key phrase “Here is my evaluation of the solution:” as well as a score within $\square$ following “Based on my evaluation, the final overall score should be:”.
- Score reward R_{score} : Rewards based on proximity between predicted score s'_i and annotated score s_i :
- 格式奖励 R_{format} : 一个指示函数, 通过检查最终回答是否包含关键短语 “Here is my evaluation of the solution:” 以及在 “Based on my evaluation, the final overall score should be:” 后面是否有一个在 $\square$ 中的分数, 来强制模型生成识别问题的摘要和一个评分。
- 评分奖励 R_{score} : 基于预测分数 s'_i 与标注分数 s_i 之间的接近程度进行奖励:

$$R_{\text{score}}(s'_i, s_i) = 1 - |s'_i - s_i| \quad (1)$$

The RL objective for training the verifier is:

训练验证器的 RL 目标是：

$$\max_{\pi_{\varphi}} \mathbb{E}_{(X_i, Y_i, s_i) \sim \mathcal{D}_{\nu}, (V'_i, s'_i) \sim \pi_{\varphi}(\cdot | X_i, Y_i)} [R_{\text{format}}(V'_i) \cdot R_{\text{score}}(s'_i, s_i)] \quad (2)$$

where V'_i denotes the verifier's final response and s'_i is the proof score extracted from it.

其中 V'_i 表示验证者的最终响应， s'_i 是从中提取的证明得分。

2.1.2. Introducing Meta-Verification to Review Proof Analyses

2.1.2. 将元验证引入以审查证明分析

The approach described in Section 2.1.1 trains proof verification through RL to align predicted proof scores with expert annotations, but provides no direct supervision on the identified issues themselves. This creates a critical vulnerability: when evaluating flawed proofs (where $s_i < 1$) during training, the verifier can receive full reward by predicting the correct scores while hallucinating non-existent issues, undermining its trustworthiness.

第 2.1.1 节中描述的方法通过强化学习训练证明验证，以使预测的证明得分与专家注释对齐，但并未对识别出的问题本身提供直接监督。这造成了一种关键的漏洞：在训练期间评估有缺陷的证明（其中 $s_i < 1$ ），验证器可以通过预测正确的分数同时虚构不存在的问题来获得全额奖励，从而损害其可信度。

To address this problem, we introduce meta-verification: a secondary evaluation process that assesses whether issues identified by the verifier indeed exist and whether these issues logically justify the predicted proof score according to the evaluation rubrics I_{ν} . The complete meta-verification rubrics I_{mv} are detailed in Appendix A.3.

为了解决这个问题，我们引入了元验证（meta-verification）：一个二级评估过程，用于评估验证器识别出的问题是否确实存在，并根据评估标准 I_{ν} 判断这些问题是否在逻辑上能够证明预测的证明分数。完整的元验证评估标准 I_{mv} 详见附录 A.3。

We trained a dedicated meta-verification using RL to perform this evaluation. By incorporating the meta-verification's feedback into verifier training, we can improve the faithfulness of the verifier's issue identification.

我们使用强化学习（RL）训练了一个专用的元验证模块，以执行此评估。通过将元验证的反馈纳入验证器的训练中，可以提高验证器识别问题的可信度。

Meta-Verifier Training Process

元验证器训练过程

1. We obtained an initial verifier π_{φ} following Section 2.1.1.
2. Mathematical experts scored the quality of verifier responses according to $\mathcal{J}_{mv}$, creating dataset $\mathcal{D}_{mv} = \{(X_i, Y_i, V_i, ms_i)\}$, where V_i is the analysis of proof Y_i and $ms_i \in \{0, 0.5, 1\}$ is the expert-annotated quality score.
3. We trained a meta-verification $\pi_{\eta}(\cdot | X, Y, V, \mathcal{J}_{mv})$ to analyze the verifier's proof analysis V . The meta-verification produces a summary of issues found in the analysis itself, followed by a quality score measuring how accurate and justified the verifier's analysis is. The RL objective follows the same structure as the verifier training, with format and score rewards.
4. 我们根据第 2.1.1 节获得了一个初始验证器 π_{φ} 。

5. 数学专家根据 J_{mv} 对验证器响应的质量进行评分，创建了数据集 $\mathcal{D}_{mv} = \{(X_i, Y_i, V_i, ms_i)\}$ ，其中 V_i 是对证明 Y_i 的分析， $ms_i \in \{0, 0.5, 1\}$ 是专家标注的质量评分。
6. 我们训练了一个元验证 $\pi_\eta(\cdot|X, Y, V, J_{mv})$ 来分析验证器的证明分析 V 。元验证会产生对分析本身所发现问题的总结，随后是一个质量评分，衡量验证器的分析准确性和合理性。强化学习目标的结构与验证器训练相同，包含格式和评分奖励。

Using the trained meta-verification π_η , we enhanced the verification by integrating meta-verification feedback into the reward function:

使用训练好的元验证 π_η ，我们通过将元验证反馈集成到奖励函数中来增强验证：

$$R_V = R_{\text{format}} \cdot R_{\text{score}} \cdot R_{\text{meta}} \quad (3)$$

where R_{meta} is the quality score from the meta-verifier.

其中 R_{meta} 是来自元验证器的质量评分。

We trained the enhanced verifier on both the verification dataset $\mathcal{D}_v$ and the meta-verification dataset $\mathcal{D}_{mv}$, using the same reward mechanism on $\mathcal{D}_{mv}$ as used for training the meta-verification. The resulting model can perform both proof verification and meta-verification tasks.

我们使用与训练元验证相同的奖励机制在 $\mathcal{D}_v$ 和元验证数据集 $\mathcal{D}_{mv}$ 上对增强的验证器进行了训练。所得到的模型可以执行证明验证和元验证任务。

On a validation split of $\mathcal{D}_v$, the average quality score of the verifier’s proof analyses - as evaluated by the meta-verification - improved from 0.85 to 0.96, while maintaining the same accuracy in proof score prediction.

在 $\mathcal{D}_v$ 的验证分割中，验证者的证明分析的平均质量得分 - 由元验证评估 - 从 0.85 提高到了 0.96，同时保持了证明得分预测的相同准确性。

2.2. Proof Generation

2.2. 证明生成

2.2.1. Training a Generator for Theorem Proving

2.2.1. 为定理证明训练生成器

With verifier π_φ serving as a generative reward model, we train a proof generator $\pi_\theta(\cdot|X)$ with the RL objective:

以验证器 π_φ 作为生成式奖励模型，我们使用强化学习目标训练一个证明生成器 $\pi_\theta(\cdot|X)$ ：

$$\max_{\pi_\theta} \mathbb{E}_{X_i \sim \mathcal{D}_p, Y_i \sim \pi_\theta(\cdot|X_i)} [R_Y] \quad (4)$$

where R_Y is the proof score produced by $\pi_\varphi(\cdot|X_i, Y_i, I_v)$.

其中 R_Y 是由 $\pi_\varphi(\cdot|X_i, Y_i, I_v)$ 产生的证明分数。

2.2.2. Enhancing Reasoning via Self-Verification

2.2.2. 通过自我验证增强推理能力

When a proof generator fails to produce a completely correct proof in one shot – common for challenging problems from competitions like IMO and CMO – iterative verification and refinement can improve results. This involves analyzing the proof with an external verifier and prompting the generator to address identified issues.

当一个证明生成器无法一次性生成完全正确的证明时——这在像 IMO 和 CMO 这样的竞赛中的挑战性问题中很常见——通过迭代验证和改进可以提高结果。这涉及使用外部验证器分析证明，并提示生成器解决已识别的问题。

However, we observed a critical limitation: when prompted to both generate and analyze its own proof in one shot, the generator tends to claim correctness even when the external verifier

然而，我们观察到一个关键的局限性：当被提示在同一时间生成并分析自己的证明时，生成器倾向于声称正确性，即使外部验证器并不认同。

easily identify flaws. In other words, while the generator can refine proofs based on external feedback, it fails to evaluate its own work with the same rigor as the dedicated verifier.

很容易发现缺陷。换句话说，虽然生成器可以根据外部反馈来完善证明，但它无法像专门的验证器那样严格地评估自己的工作。

This observation motivated us to endow the proof generator with genuine verification capabilities. During training, we prompt the generator π_θ to produce a proof Y followed by a self-analysis Z that follows the same format and rubrics $\mathcal{J}_\nu$ as the verifier (see Appendix A.1). We denote the proof score predicted in the self-analysis as s' .

这一观察促使我们赋予证明生成器真实的验证能力。在训练过程中，我们提示生成器 π_θ 生成一个证明 Y ，随后是遵循与验证器相同格式和评分标准 $\mathcal{J}_\nu$ 的自我分析 Z （参见附录 A.1）。我们将自我分析中预测的证明得分表示为 s' 。

To ensure faithful self-evaluation, we use the verifier π_φ to assess both components: the proof Y receives score $R_Y = s$, and the self-analysis Z receives a meta-verification score $R_{\text{meta}}(Z) = ms$. The reward function combines these assessments:

为确保忠实的自我评估，我们使用验证器 π_φ 来评估两个组件：证明 Y 获得分数 $R_Y = s$ ，而自我分析 Z 获得元验证分数 $R_{\text{meta}}(Z) = ms$ 。奖励函数结合了这些评估：

$$R = R_{\text{format}}(Y, Z) \cdot (\alpha \cdot R_Y + \beta \cdot R_Z) \quad (5)$$

$$R_Z = R_{\text{score}}(s', s) \cdot R_{\text{meta}}(Z) \quad (6)$$

where $R_{\text{format}}(Y, Z)$ verifies that both the proof and self-analysis follow the specified format, $R_{\text{score}}(s', s)$ rewards accurate self-assessment. We set $\alpha = 0.76$ and $\beta = 0.24$. This reward structure creates the following incentives:

where $R_{\text{format}}(Y, Z)$ verifies that both the proof and self-analysis follow the specified format, $R_{\text{score}}(s', s)$ rewards accurate self-assessment. We set $\alpha = 0.76$ and $\beta = 0.24$. This reward structure creates the following incentives:

- Faithful acknowledgment of errors is rewarded over false claims of correctness.
- The highest rewards come from producing correct proofs and accurately recognizing their rigor.
- A good strategy to obtain high rewards for the proof generator is to identify and resolve as many issues as possible before finalizing the response.

- 对错误的忠实承认会比虚假的正确性声明获得更高的奖励。
- 最高的奖励来自于产生正确的证明并准确识别其严谨性。
- 获得证明生成器高奖励的一个好策略是在最终回答前尽可能识别和解决更多的问题。

2.3. Synergy Between Proof Verification and Generation

2.3. 证明验证与生成之间的协同作用

The proof verifier and generator create a synergistic cycle: the verifier improves the generator, and as the generator improves, it produces new proofs that challenge the verifier's current capabilities. These challenging cases – where the verifier may fail to identify issues in a single attempt – become valuable training data for enhancing the verifier itself.

证明验证器和生成器创建了一个协同循环：验证器改进生成器，而随着生成器的改进，它会产生新的证明，挑战验证器当前的能力。这些具有挑战性的案例——在这些案例中，验证器可能在一次尝试中无法识别问题——成为增强验证器本身的宝贵训练数据。

To retrain and improve the verifier, we need labeled correctness data for newly generated proofs. Manual annotation, while straightforward, becomes increasingly time-consuming as problems grow harder and errors become more subtle. To boost annotation efficiency, we generated multiple verifier analyses per proof to surface potential issues for human review.

为了重新训练和改进验证器，我们需要为新生成的证明提供带有标签的正确性数据。虽然手动标注相对直接，但随着问题变得越来越复杂且错误更加微妙，标注工作会变得越来越耗时。为了提高标注效率，我们为每个证明生成了多个验证器分析，以揭示潜在问题供人类审查。

From this AI-assisted annotation process, we recognized two facts that make it feasible to push the level of automation a step further:

从这个由人工智能辅助的注释过程中，我们认识到两个事实，使得进一步提升自动化水平成为可能：

1. Scaling verifier samples increases the probability of catching real issues in flawed proofs.
2. Reviewing the verifier's identified issues is exactly meta-verification, which is easier than identifying issues from scratch. Meta-verification is also more sample-efficient for LLMs to master.
3. 增加验证器样本的数量可以提高在有缺陷的证明中发现真实问题的概率。
4. 审查验证器识别出的问题正是元验证，这比从头开始识别问题要容易。对于 LLMs 来说，元验证也是更高效的学习方式。

Building on these observations, we developed the following automated labeling process:

基于这些观察，我们开发了以下自动标注流程：

1. For each proof, generate n independent verification analyses
2. 对于每个证明，生成 n 独立的验证分析
3. For analyses reporting issues (scores 0 or 0.5), generate m meta-verification assessments to validate the identified problems. An analysis is deemed valid if the majority of meta-assessments confirm its findings
4. For each proof, we examine analyses that assign the lowest score. If at least k such analyses are deemed valid, the proof is labeled with that lowest score. If no legitimate issues are identified across all verification attempts, the proof is labeled with 1. Otherwise, the proof is discarded or routed to human experts for labeling

5. 对于报告问题的分析（得分 0 或 0.5），生成 m 元验证评估以验证识别出的问题。如果大多数元评估确认了分析的结果，则该分析被视为有效
6. 对于每个证明，我们检查分配最低分数的分析。如果至少 k 这样的分析被判定为有效，则该证明将标记为该最低分数。如果在所有验证尝试中均未识别出任何合法问题，则该证明将标记为 1。否则，该证明将被丢弃或转交给人类专家进行标记

In our last two training iterations, this fully automated pipeline replaced human annotation entirely. Quality checks confirmed that the automated labels aligned well with expert judgments.

在我们最近的两次训练迭代中，这个完全自动化的流程完全取代了人工标注。质量检查确认了自动标注结果与专家判断高度一致。

3. Experiments

3. 实验

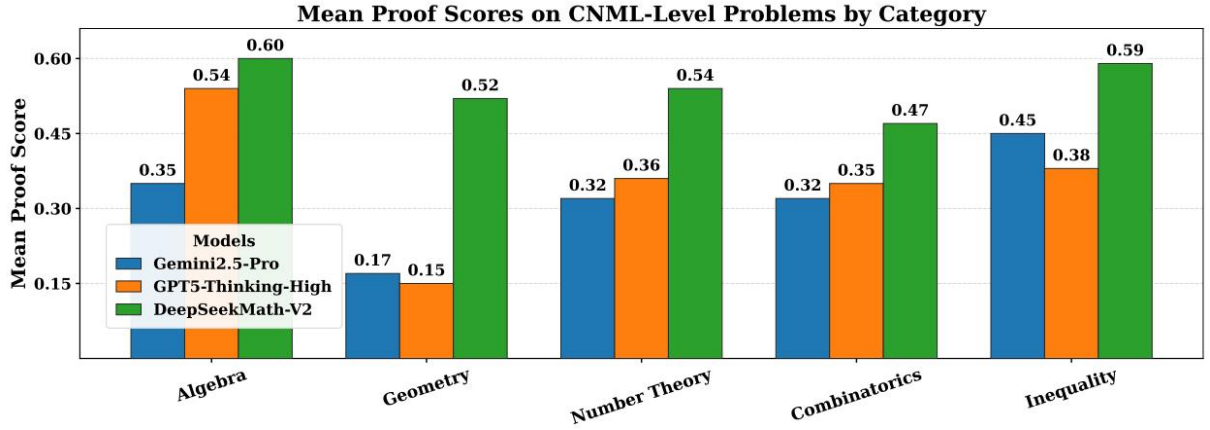


Figure 1 | Average proof scores on CNML-level problems by category and model, as evaluated by our verifier.

图 1 | 我们的验证器评估的按类别和模型在 CNML 级别问题上的平均证明得分。

3.1. Training Settings

3.1. 训练设置

We employed Group Relative Policy Optimization (GRPO) (Shao et al., 2024) for reinforcement learning, iteratively optimizing proof verification and generation capabilities as described in Section 2. In each iteration, we first optimized proof verification. The proof generator was then initialized from the verifier checkpoint and optimized for proof generation. Starting from the second iteration, the proof verifier was initialized with a checkpoint that consolidated both verification and generation capabilities from the previous iteration through rejection fine-tuning.

我们采用了组相对策略优化（GRPO）（Shao 等，2024）来进行强化学习，如第 2 节所述，迭代地优化证明验证和生成能力。在每次迭代中，我们首先优化证明验证。然后，从验证器检查点初始化证明生成器，并对其进行证明生成的优化。从第二次迭代开始，证明验证器使用一个检查点进行初始化，该检查点通过拒绝微调，整合了上一次迭代中验证和生成能力。

3.2. Evaluation Benchmarks

3.2. 评估基准

We evaluate our final proof generator on the following theorem proving benchmarks:

我们在以下定理证明基准上评估我们的最终证明生成器：

In-House CNML-Level Problems 91 theorem-proving problems spanning algebra (13), geometry (24), number theory (19), combinatorics (24), and inequality (11), comparable in difficulty to problems from Chinese National High School Mathematics League (CNML)

内部命题 CNML 级问题 91 道定理证明题，涵盖代数（13）、几何（24）、数论（19）、组合数学（24）和不等式（11），难度相当于中国全国高中数学联合竞赛（CNML）的题目

Competition Problems

竞赛题目

- IMO 2025 (6 problems): The International Mathematical Olympiad, the premier global mathematics competition for pre-university students
- IMO 2025 (6 problems): 国际数学奥林匹克，这是为预科学生举办的全球最高水平的数学竞赛
- CMO 2024 (6 problems): The China Mathematical Olympiad, China's national championship
- Putnam 2024 (12 problems): The William Lowell Putnam Competition, the preeminent mathematics competition for undergraduate students in North America
- ISL 2024 (31 problems): The IMO Shortlist, a collection of problems proposed by participating countries and considered by the Problem Selection Committee for potential inclusion in IMO 2024
- IMO-ProofBench (60 problems): Developed by the DeepMind team behind DeepThink IMO-Gold (Luong and Lockhart, 2025), this benchmark (Luong et al., 2025) is divided into a basic set (30 problems, pre-IMO to IMO-Medium difficulty) and an advanced set (30 challenging problems simulating complete IMO examinations, up to IMO-Hard level)
- CMO 2024 (6 problems): 中国数学奥林匹克，中国的全国性锦标赛
- Putnam 2024 (12 problems): 威廉·洛厄尔·普特南数学竞赛，北美本科生最负盛名的数学竞赛
- ISL 2024 (31 problems): 国际数学奥林匹克短名单，由参赛国家提出的题目集合，由问题选择委员会考虑是否纳入 2024 年国际数学奥林匹克
- IMO-ProofBench (60 problems): 由 DeepThink IMO-Gold (Luong 和 Lockhart, 2025) 背后的 DeepMind 团队开发，该基准 (Luong 等, 2025) 分为基础集 (30 题，难度从预 IMO 到 IMO-中等) 和高级集 (30 道具有挑战性的题目，模拟完整的 IMO 考试，难度达到 IMO-困难级别)

3.3. Evaluation Results

3.3. 评估结果

3.3.1. One-Shot Generation

3.3.1. 一次性生成

We first evaluate the model’s ability to generate correct proofs without iterative refinement. On the in-house problems, we generated 8 proof samples per problem for each evaluated model. Proof correctness was measured by majority voting across 8 verification analyses produced by our final verifier. As shown in Figure 1, across all categories of CNML-level problems – algebra, geometry, number theory, combinatorics, and inequality – DeepSeekMath-V2 consistently outperforms GPT-5-Thinking-High (OpenAI, 2025) and Gemini 2.5-Pro (DeepMind, 2025), demonstrating superior theorem-proving ability across domains.

我们首先评估模型在不进行迭代优化的情况下生成正确证明的能力。在内部问题上，我们为每个评估模型生成了每个问题的 8 个证明样本。证明的正确性通过我们最终验证器生成的 8 个验证分析的多数投票来衡量。如图 1 所示，在所有 CNML 级别问题的类别——代数、几何、数论、组合数学和不等式——中，DeepSeekMath-V2 始终优于 GPT-5-Thinking-High (OpenAI, 2025) 和 Gemini 2.5-Pro (DeepMind, 2025)，展示了其在各个领域卓越的定理证明能力。

3.3.2. Sequential Refinement with Self-Verification

3.3.2. 顺序优化与自我验证

For challenging problems from competitions like IMO and CMO, models often cannot generate comprehensive and rigorous proofs in a single attempt within the 128K token limit. When this occurs, our proof generator recognizes its proof is invalid through self-verification but lacks the context length to resolve all identified issues in a single attempt.

对于来自 IMO 和 CMO 等竞赛中的难题，模型通常无法在 128K token 的限制内一次生成全面且严谨的证明。当这种情况发生时，我们的证明生成器会通过自我验证识别其证明无效，但缺乏上下文长度来在一次尝试中解决所有识别出的问题。

To explore how extended context and self-verification can improve proof quality, we evaluate sequential refinement with self-verification. This approach first generates a proof with self-analysis, then iteratively re-prompts the generator with its previous output (see Appendix A.4 for the refinement prompt), allowing it to address

为了探讨扩展上下文和自我验证如何提高证明质量，我们评估了结合自我验证的序列细化方法。这种方法首先通过自我分析生成一个证明，然后迭代地将生成器重新提示其之前的输出（参见附录 A.4 中的细化提示），从而使其能够解决

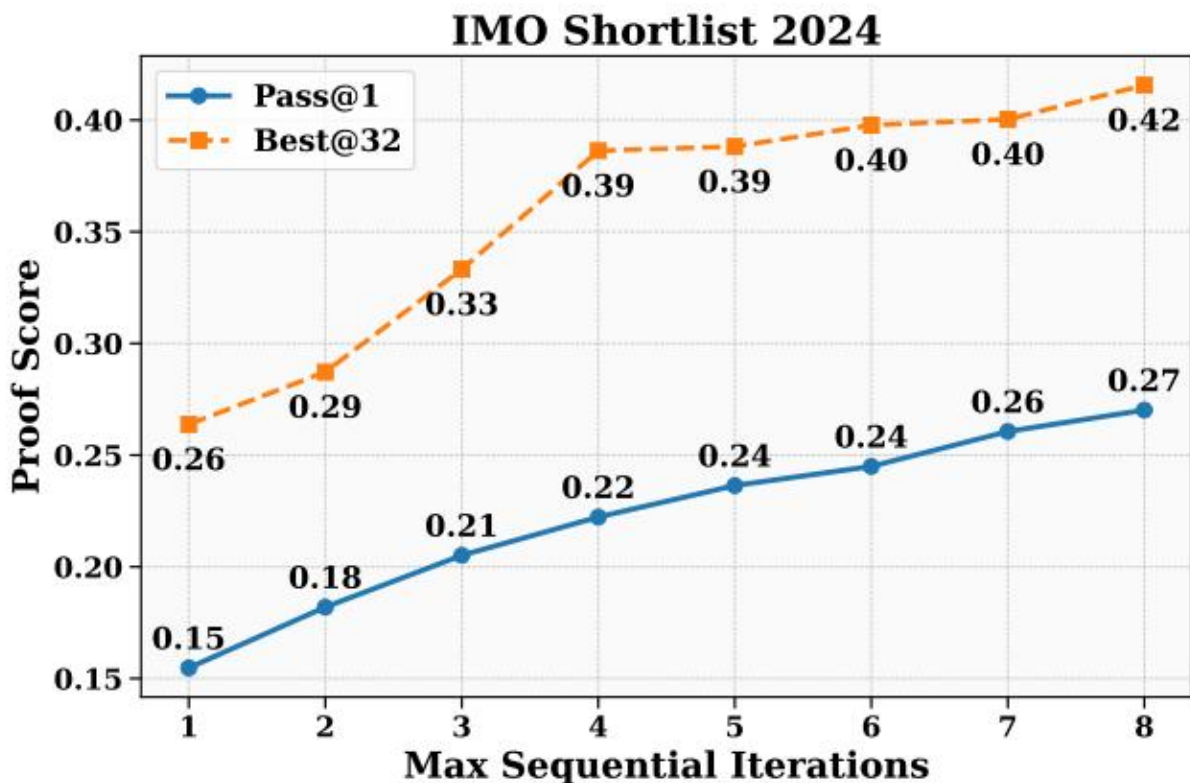


Figure 2 | Proof quality improvements as the maximum sequential iterations varies from 1 (no refinement) to 8 (initial generation plus up to 7 refinements based on self verification).

图 2 | 证明质量的改进随着最大连续迭代次数从 1（无优化）变化到 8（初始生成加上最多 7 次基于自我验证的优化）。

identified issues. The process continues until the generator assigns itself a perfect score or reaches the maximum number of sequential attempts.

已识别的问题。该过程持续进行，直到生成器为自己分配一个完美分数或达到最大连续尝试次数。

Figure 2 demonstrates proof quality improvement through sequential refinement on IMO Shortlist 2024 problems. For each problem, we launched 32 independent refinement threads. Proof correctness was measured by majority voting across 32 verification analyses from our final verifier. We report two metrics in Figure 2: (1) Pass@1 – the average score of the final

图 2 展示了通过顺序细化在 IMO 2024 短名单问题上证明质量的提升。对于每个问题，我们启动了 32 个独立的细化线程。证明的正确性通过我们最终验证器的 32 次验证分析中的多数投票来衡量。我们在图 2 中报告了两个指标：(1) Pass@1 – 最终验证分析的平均得分

proof from each thread, and (2) Best@32 – the score of the best proof per problem, selected by self-assigned scores across all threads. The self-selected best proofs achieve significantly higher verification scores than the thread average, demonstrating our generator’s ability to accurately assess proof quality. Furthermore, Pass@1 improves substantially as maximum sequential attempts increase, showing that self-verification effectively guides iterative improvement. These results confirm that our generator can reliably differentiate between high-quality and flawed proofs, and leverage this self-awareness to systematically improve its mathematical reasoning.

来自每个线程的证明，以及 (2) Best@32 – 每个问题中自选的最佳证明得分，通过所有线程中的自评分进行选择。自选的最佳证明在验证得分上显著高于线程平均值，这表明我们的生成器能够准确评估证明的质量。此外，随着最大连续尝试次数的增加，Pass@1 显著提高，这表明自我验证有效地引导了迭代改进。这些结果证实了我们的生成器能够可靠地区分高质量和有缺陷的证明，并利用这种自我意识系统地提高其数学推理能力。

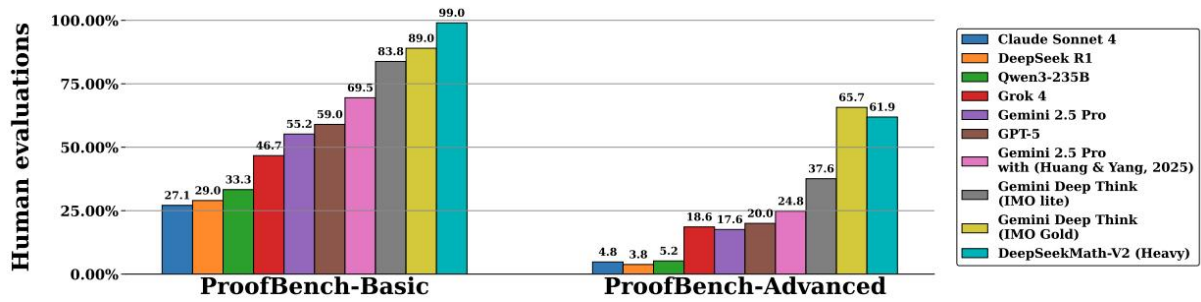


Figure 3 | Expert evaluation results on the Basic and Advanced subsets of IMO-ProofBench. All results are sourced from Luong et al. (2025), with the exception of DeepSeekMath-V2, which was evaluated by our experts following the grading guidelines.

图 3 | 在 IMO-ProofBench 的基本集和高级集上的专家评估结果。所有结果均来源于 Luong 等人 (2025)，除 DeepSeekMath-V2 外，该模型的评估是由我们专家根据评分指南进行的。

3.3.3. High-Compute Search

3.3.3. 高计算搜索

To solve the most challenging problems, we scaled both verification and generation compute - using extensive verification to identify subtle issues and parallel generation to explore diverse proof strategies.

为了解决最具挑战性的问题，我们扩展了验证和生成的计算能力——使用广泛的验证来识别细微的问题，并通过并行生成来探索多样化的证明策略。

Our approach maintains a pool of candidate proofs for each problem, initialized with 64 proof samples with 64 verification analyses generated for each. In each refinement iteration, we select the 64 highest-scoring proofs based on average verifica

我们的方法为每个问题维护一个候选证明池，初始时包含 64 个证明样本，每个样本都生成了 64 个验证分析。在每次细化迭代中，我们根据平均验证得分选择 64 个最高分的证明。

Table 1 | Problems in gray are fully solved, while underlined problems received partial credit.

表 1 | 灰色问题已完全解决，而下划线问题获得部分分数。

Contest	Problems					Points
IMO 2025	P1,	P2,	P3,	P4,	P5	83.3%
CMO 2024	P1,	P2,	P4,	P5,	P6	73.8%
Putnam 2024	A1 ~	B4,	B5,	B6		98.3%

比赛	问题					要点
IMO 2025	P1,	P2,	P3,	P4,	P5	83.3%
CMO 2024	P1,	P2,	P4,	P5,	P6	73.8%
Putnam 2024	A1 ~	B4,	B5,	B6		98.3%

tion scores and pair each with 8 randomly selected analyses, prioritizing those identifying issues (scores 0 or 0.5). Each proof-analysis pair is used to generate one refined proof, which then updates the candidate pool. This process continues for up to 16 iterations or until a proof successfully passes all 64 verification attempts, indicating high confidence in correctness. All experiments used a single model, our final proof generator, which performs both proof generation and verification.

评分并将其与 8 个随机选择的分析进行配对，优先选择那些识别出问题的分析（评分 0 或 0.5）。每个证明-分析对用于生成一个改进后的证明，然后更新候选池。此过程最多进行 16 次迭代，或直到一个证明成功通过所有 64 次验证尝试，表明其正确性具有高度信心。所有实验都使用了一个模型，即我们的最终证明生成器，它同时执行证明生成和验证。

To validate our results, mathematical experts assessed the highest-scoring proofs. As shown in Table 1, our approach solved 5 of 6 problems from IMO 2025 and 4 problems plus partial credit on another from CMO 2024, achieving gold medal performance in both pinnacle high-school competitions. On Putnam 2024, the preeminent undergraduate mathematics competition, our model solved 11 of 12 problems completely and the remaining problem with minor errors, scoring 118/120 and surpassing the highest human score of 90. Figure 3 shows the results on IMO-ProofBench. Our approach outperforms DeepMind’s DeepThink (IMO Gold) on the basic set and remains competitive on the advanced set, while substantially outperforming all other baselines. We observe that the hardest IMO-level problems remain challenging for our model.

为了验证我们的结果，数学专家评估了得分最高的证明。如表 1 所示，我们的方法在 IMO 2025 中解决了 6 个问题中的 5 个，并在 CMO 2024 中解决了 4 个问题并获得了另一个问题的部分分数，从而在两项顶尖的高中数学竞赛中均取得了金牌成绩。在 Putnam 2024（顶尖的大学数学竞赛）中，我们的模型完全解决了 12 个问题中的 11 个，并在最后一个问题上出现了小错误，得分为 118/120，超过了人类最高得分 90。图 3 展示了我们在 IMO-ProofBench 上的结果。我们的方法在基础集上优于 DeepMind 的 DeepThink (IMO 金牌)，在高级集上仍保持竞争力，同时显著优于所有其他基线。我们观察到，最难的 IMO 级别问题对我们的模型来说仍然具有挑战性。

Notably, for problems not fully solved, our generator typically identifies the genuine issues in its proofs, while fully solved problems pass all 64 verification attempts. This demonstrates that we can successfully train LLM-based verifiers to assess proofs previously considered difficult to verify automatically. By scaling test-time compute under verifier guidance, our model solves problems that require hours of effort from human competitors.

值得注意的是，对于尚未完全解决的问题，我们的生成器通常能够在其证明中识别出真正的难点，而完全解决的问题则能通过全部 64 次验证尝试。这表明我们可以成功地训练基于 LLM 的验证器，以自动评估此前被认为难以验证的证明。通过在验证器指导下扩展测试时的计算资源，我们的模型能够解决需要人类竞争者数小时努力的问题。

4. Related Work

4. 相关工作

Reasoning models (OpenAI, 2024; Guo et al., 2025) have saturated quantitative reasoning benchmarks like AIME and HMMT within one year. This rapid advancement is partly attributed to the well-defined evaluation criterion: if we care only about final answers, then quantitative reasoning is easy to verify. However, this final answer metric is inapplicable to theorem proving, which often requires no numerical answers but demands rigorous step-by-step derivation. Informal mathematical proofs have long been considered hard to verify automatically, lacking reliable approaches to assess proof correctness. Recent developments suggest this barrier may be surmountable. Models like Gemini-2.5 Pro already demonstrate a certain level of self-verification capabilities, which can refine their own solutions to improve quality (Huang and Yang, 2025). More significantly, DeepMind’s internal DeepThink variant (Luong and Lockhart, 2025) achieved gold medal performance at IMO 2025 using pure natural language reasoning – serving as an existence proof that LLM-based verification of complex proofs is achievable. Recent research has begun exploring whether reasoning models can evaluate proofs, both with and without reference solutions (Dekoninck et al., 2025; Luong et al., 2025), showing promising results. In this work, we open source DeepSeekMath-V2 and our training methodology as concrete steps toward self-verifiable mathematical reasoning, showing how models can learn to verify and improve their own proofs.

推理模型 (OpenAI, 2024; Guo 等人, 2025) 在一年内就使 AIME 和 HMMT 等定量推理基准测试趋于饱和。这种快速进步在一定程度上归因于明确的评估标准：如果我们只关注最终答案，那么定量推理就很容易验证。然而，这种最终答案指标不适用于定理证明，因为定理证明通常不需要数值答案，而是要求严谨

的逐步推导。非正式的数学证明长期以来被认为难以自动验证，缺乏可靠的方法来评估证明的正确性。最近的发展表明，这一障碍可能是可以克服的。像 Gemini-2.5 Pro 这样的模型已经展示了某种程度的自我验证能力，可以改进自身的解决方案以提高质量 (Huang 和 Yang, 2025)。更重要的是，DeepMind 内部的 DeepThink 变体 (Luong 和 Lockhart, 2025) 使用纯粹的自然语言推理在 IMO 2025 中取得了金牌成绩——这证明了基于 LLM 的复杂证明验证是可行的。最近的研究开始探索推理模型是否能够评估证明，无论是否有参考解 (Dekoninck 等人, 2025; Luong 等人, 2025)，显示出有希望的结果。在本工作中，我们开源了 DeepSeekMath-V2 和我们的训练方法，作为实现可自我验证的数学推理的具体步骤，展示了模型如何学习验证并改进自身的证明。

Proof assistants like Lean (de Moura et al., 2015) and Isabelle (Paulson, 1994) offer a reliable approach to verify proofs – proofs must be written in formal language, but once compiled, correctness is guaranteed. AlphaProof (AlphaProof and teams, 2024; Trinh et al., 2024; Chervonyi et al., 2025), a system specialized for formal proof search, achieved silver-level performance at IMO 2024 but required intensive computation. While using informal reasoning to guide formal proof generation has been explored extensively (Jiang et al., 2023), recent reasoning models have dramatically improved informal reasoning quality, making this guidance far more effective. Systems like DeepSeek-Prover-V2 (Ren et al., 2025) and Seed-Prover (Chen et al., 2025) can now produce substantially more valid formal proofs within the same computational budget, with Seed-Prover solving 5 of 6 problems at IMO 2025. Notably, these results were achieved without specifically optimizing the informal reasoning components for theorem proving tasks. We believe advancing natural language theorem proving will significantly benefit formal reasoning. We hope to contribute toward truly reliable mathematical reasoning systems that leverage both informal insights and formal guarantees to advance mathematical research.

像 Lean (de Moura 等, 2015) 和 Isabelle (Paulson, 1994) 这样的证明助手提供了一种可靠的方法来验证证明——证明必须用形式语言编写，但一旦编译，正确性就得到了保证。AlphaProof (AlphaProof 和团队, 2024; Trinh 等, 2024; Chervonyi 等, 2025) 是一个专门用于形式证明搜索的系统，在 IMO 2024 中达到了银级表现，但需要大量的计算。虽然使用非正式推理来指导形式证明生成已经被广泛探索 (Jiang 等, 2023)，但最近的推理模型极大地提高了非正式推理的质量，使得这种指导远更为有效。像 DeepSeek-Prover-V2 (Ren 等, 2025) 和 Seed-Prover (Chen 等, 2025) 这样的系统现在可以在相同的计算预算内生成大量有效的形式证明，Seed-Prover 在 IMO 2025 中解决了 6 个问题中的 5 个。值得注意的是，这些结果是在没有特别针对定理证明任务优化非正式推理组件的情况下取得的。我们认为，推进自然语言定理证明将显著促进形式推理的发展。我们希望为真正可靠的数学推理系统做出贡献，这些系统能够利用非正式洞察和形式保证来推进数学研究。

5. Conclusion

5. 结论

We presented DeepSeekMath-V2, a model capable of both generating and verifying mathematical proofs. By training models to identify issues in their own reasoning and incentivizing them to address these issues before finalizing outputs, we move beyond the limitations of final-answer-based rewards toward self-verifiable mathematical reasoning. Our iterative training process – alternating between improving verification capabilities and using these to enhance generation – creates a sustainable cycle where each component drives the other forward. Our

我们提出了 DeepSeekMath-V2，这是一个能够生成和验证数学证明的模型。通过训练模型识别自身推理中的问题，并激励它们在最终输出前解决这些问题，我们超越了基于最终答案的奖励机制的局限性，转向可自我验证的数学推理。我们的迭代训练过程——在提升验证能力与利用这些能力来增强生成之间交替进行——创造了一个可持续的循环，其中每个组件都推动另一个组件向前发展。我们的

key technical contributions include: (1) training an accurate and faithful LLM-based verifier for mathematical proofs, (2) using meta-verification to largely reduce hallucinated issues and ensure verification quality, (3) incentivizing the proof generator to maximize proof quality through self-verification, and (4) scaling verification compute to automatically label increasingly hard-to-verify proofs to improve the verifier without human annotation. DeepSeekMath-V2 demonstrates strong performance on competition mathematics. With scaled test-time compute, it achieved gold-medal scores in high-school competitions including IMO 2025 and CMO 2024, and a near-perfect score on the undergraduate Putnam 2024 competition. This work establishes

that LLMs can develop meaningful self-evaluation abilities for complex reasoning tasks. While significant challenges remain, we hope this research direction contributes to the goal of creating self-verifiable AI systems that can solve research-level mathematics.

主要技术贡献包括：(1) 训练一个准确且忠实的基于大语言模型（LLM）的验证器用于数学证明，(2) 使用元验证（meta-verification）大幅减少幻觉问题并确保验证质量，(3) 通过自我验证激励证明生成器以最大化证明质量，(4) 扩展验证计算能力以自动标记越来越难以验证的证明，从而在无需人工标注的情况下提升验证器性能。DeepSeekMath-V2 在竞赛数学上表现出色。通过扩展测试时的计算能力，它在包括 2025 年国际数学奥林匹克（IMO）和 2024 年中国数学奥林匹克（CMO）在内的高中竞赛中取得了金牌成绩，并在 2024 年本科生普林斯顿数学竞赛（Putnam）中取得了接近完美的成绩。本工作证明了大语言模型可以发展出对复杂推理任务有意义的自我评估能力。尽管仍存在重大挑战，我们希望这一研究方向有助于实现能够解决研究级数学的可自我验证的 AI 系统的目标。

References

参考资料

AlphaProof and A. teams. Ai achieves silver-medal standard solving international mathematical olympiad problems, 2024. URL <https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level>.

L. Chen, J. Gu, L. Huang, W. Huang, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, K. Ma, C. Ren, J. Shen, W. Shi, T. Sun, H. Sun, J. Wang, S. Wang, Z. Wang, C. Wei, S. Wei, Y. Wu, Y. Wu, Y. Xia, H. Xin, F. Yang, H. Ying, H. Yuan, Z. Yuan, T. Zhan, C. Zhang, Y. Zhang, G. Zhang, T. Zhao, J. Zhao, Y. Zhou, and T. H. Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving. CoRR, abs/2507.23726, 2025. doi: 10.48550/ARXIV.2507.23726. URL <https://doi.org/10.48550/arXiv.2507.23726>.

Y. Chervonyi, T. H. Trinh, M. Olsák, X. Yang, H. Nguyen, M. Menegali, J. Jung, V. Verma, Q. V. Le, and T. Luong. Gold-medalist performance in solving olympiad geometry with alphageometry2. CoRR, abs/2502.03544, 2025. doi: 10.48550/ARXIV.2502.03544. URL <https://doi.org/10.48550/arXiv.2502.03544>.

L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The lean theorem prover (system description). In A. P. Felty and A. Middeldorp, editors, Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings, volume 9195 of Lecture Notes in Computer Science, pages 378-388. Springer, 2015. doi: 10.1007/978-3-319-21401-6_26. URL https://doi.org/10.1007/978-3-319-21401-6_26.

G. DeepMind. Gemini 2.5 pro, 2025. URL <https://deepmind.google/models/gemini/pro>.

DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.

J. Dekoninck, I. Petrov, K. Minchev, M. Balunovic, M. T. Vechev, M. Marinov, M. Drencheva, L. Konova, M. Shumanov, K. Tsvetkov, N. Drenchev, L. Todorov, K. Nikolova, N. Georgiev, V. Kalinkova, and M. Ismoldayev. The open proof corpus: A large-scale study of llm-generated mathematical proofs. CoRR, abs/2506.21621, 2025. doi: 10.48550/ARXIV.2506.21621. URL <https://doi.org/10.48550/arXiv.2506.21621>.

D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng,

AlphaProof 和 A. 团队. Ai 在 2024 年达到银牌水平，解决国际数学奥林匹克问题。URL <https://deepmind.google/blog/ai-solves-imo-problems-at-silver-medal-level>.

L. Chen, J. Gu, L. Huang, W. Huang, Z. Jiang, A. Jie, X. Jin, X. Jin, C. Li, K. Ma, C. Ren, J. Shen, W. Shi, T. Sun, H. Sun, J. Wang, S. Wang, Z. Wang, C. Wei, S. Wei, Y. Wu, Y. Wu, Y. Xia, H. Xin, F. Yang, H. Ying, H. Yuan, Z. Yuan, T. Zhan, C. Zhang, Y. Zhang, G. Zhang, T. Zhao, J. Zhao, Y. Zhou, and T. H. Zhu. Seed-prover: Deep and broad reasoning for automated theorem proving. CoRR, abs/2507.23726, 2025. doi: 10.48550/ARXIV.2507.23726. URL <https://doi.org/10.48550/arXiv.2507.23726>.

Y. Chervonyi, T. H. Trinh, M. Olsák, X. Yang, H. Nguyen, M. Menegali, J. Jung, V. Verma, Q. V. Le, and T. Luong. Gold-medalist performance in solving olympiad geometry with alphageometry2. CoRR, abs/2502.03544, 2025. doi: 10.48550/ARXIV.2502.03544. URL <https://doi.org/10.48550/arXiv.2502.03544>.

L. M. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer. The lean theorem prover

(system description). In A. P. Felty and A. Middeldorp, editors, Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings, volume 9195 of Lecture Notes in Computer Science, pages 378-388. Springer, 2015. doi: 10.1007/978-3-319-21401-6_26. URL https://doi.org/10.1007/978-3-319-21401-6_26.

G. DeepMind. Gemini 2.5 pro, 2025. URL <https://deepmind.google/models/gemini/pro>.

DeepSeek-AI. Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention, 2025.

J. Dekoninck, I. Petrov, K. Minchev, M. Balunovic, M. T. Vechev, M. Marinov, M. Drencheva, L. Konova, M. Shumanov, K. Tsvetkov, N. Drenchev, L. Todorov, K. Nikolova, N. Georgiev, V. Kalinkova, and M. Ismoldayev. The open proof corpus: A large-scale study of llm-generated mathematical proofs. CoRR, abs/2506.21621, 2025. doi: 10.48550/ARXIV.2506.21621. URL <https://doi.org/10.48550/arXiv.2506.21621>.

D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng,

C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z.Z.RenZ.RenZ.ShaZ.FuZ.XuZ.Xie Z.Zhang,Z.HaoZ.MaZ.YanZ.WuZ.GuZ.ZhuZ.LiuZ.LiZ.XieZ.SongZ.Pan Z.HuangZ.XuZ.Zhang and Z.Zhang.Deepseek-r1 incentivizes reasoning in llms through reinforcement learning.Nat.,645(8081):633-6382025.doi:10.1038/S41586-025-09422-Z URL <https://doi.org/10.1038/s41586-025-09422-z>

Y. Huang and L. F. Yang. Gemini 2.5 pro capable of winning gold at IMO 2025. CoRR, abs/2507.15855, 2025. doi: 10.48550/ARXIV.2507.15855. URL <https://doi.org/10.48550/arXiv.2507.15855>.

A. Q. Jiang, S. Welleck, J. P. Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023. OpenReview.net, 2023. URL <https://openreview.net/forum?id=SMA9EAovKMC>.

T. Luong and E. Lockhart. Advanced version of gemini with deep think officially achieves gold-medal standard at the international mathematical olympiad, 2025. URL <https://goo.gl/imo-gold>.

T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. Towards robust mathematical reasoning. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, 2025. URL: <https://aclanthology.org/2025.emnlp-main.1794/>.

OpenAI. Learning to reason with llms, 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.

OpenAI. Introducing gpt-5, 2025. URL <https://openai.com/index/introducing-gpt-5>.

L. C. Paulson. Isabelle - A Generic Theorem Prover (with a contribution by T. Nipkow), volume 828 of Lecture Notes in Computer Science. Springer, 1994. ISBN 3-540-58244-4. doi: 10.1007/BFB0030541. URL <https://doi.org/10.1007/BFB0030541>.

Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, Z. F. Wu, Z. Gou, S. Ma, H. Tang, Y. Liu, W. Gao, D. Guo, and C. Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition. CoRR, abs/2504.21801, 2025. doi: 10.48550/ARXIV.2504.21801. URL <https://doi.org/10.48550/arXiv.2504.21801>.

C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K.

Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z.Z.RenZ.RenZ.ShaZ.FuZ.XuZ.Xie Z.Zhang,Z.HaoZ.MaZ.YanZ.WuZ.GuZ.ZhuZ.LiuZ.LiZ.XieZ.SongZ.Pan Z.HuangZ.XuZ.Zhang and Z.Zhang.Deepseek-r1 通过强化学习激励大型语言模型中的推理. *Nat.*,645(8081):633-6382025.doi:10.1038/S41586-025-09422-Z URL <https://doi.org/10.1038/s41586-025-09422-z>

Y. Huang and L. F. Yang. Gemini 2.5 pro 能够赢得 2025 年 IMO 金牌. *CoRR*, abs/2507.15855, 2025. doi: 10.48550/ARXIV.2507.15855. URL <https://doi.org/10.48550/arXiv.2507.15855>.

A. Q. Jiang, S. Welleck, J. P. Zhou, T. Lacroix, J. Liu, W. Li, M. Jamnik, G. Lample, and Y. Wu. Draft, sketch, and prove: 通过非正式证明指导形式定理证明器. 在第十一届国际学习表示会议, ICLR 2023, 卢旺达基加利, 2023 年 5 月 1 日至 5 日. OpenReview.net, 2023. URL <https://openreview.net/forum?id=SMA9EAovKMC>.

T. Luong and E. Lockhart. 具有深度思考的 Gemini 高级版本正式在国际数学奥林匹克比赛中达到金牌标准, 2025. URL <https://goo.gl/imo-gold>.

T. Luong, D. Hwang, H. H. Nguyen, G. Ghiasi, Y. Chervonyi, I. Seo, J. Kim, G. Bingham, J. Lee, S. Mishra, A. Zhai, C. H. Hu, H. Michalewski, J. Kim, J. Ahn, J. Bae, X. Song, T. H. Trinh, Q. V. Le, and J. Jung. 向鲁棒数学推理迈进. 在 2025 年经验方法自然语言处理会议论文集, 2025. URL: <https://aclanthology.org/2025.emnlp-main.1794/>.

OpenAI. 通过大型语言模型学习推理, 2024. URL <https://openai.com/index/learning-to-reason-with-llms/>.

OpenAI. 引入 GPT-5, 2025. URL <https://openai.com/index/introducing-gpt-5>.

L. C. Paulson. Isabelle - 一个通用的定理证明器 (由 T. Nipkow 贡献), 计算机科学讲义第 828 卷. Springer, 1994. ISBN 3-540-58244-4. doi: 10.1007/BFB0030541. URL <https://doi.org/10.1007/BFB0030541>.

Z. Z. Ren, Z. Shao, J. Song, H. Xin, H. Wang, W. Zhao, L. Zhang, Z. Fu, Q. Zhu, D. Yang, Z. F. Wu, Z. Gou, S. Ma, H. Tang, Y. Liu, W. Gao, D. Guo, and C. Ruan. Deepseek-prover-v2: 通过强化学习进行子目标分解以推进形式数学推理. *CoRR*, abs/2504.21801, 2025. doi: 10.48550/ARXIV.2504.21801. URL <https://doi.org/10.48550/arXiv.2504.21801>.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL <https://doi.org/10.48550/arXiv.2402.03300>.

T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nat.*, 625(7995):476-482, 2024. doi: 10.1038/S41586-023-06747-5. URL <https://doi.org/10.1038/s41586-023-06747-5>.

Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, M. Zhang, Y. K. Li, Y. Wu, and D. Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300, 2024. doi: 10.48550/ARXIV.2402.03300. URL <https://doi.org/10.48550/arXiv.2402.03300>.

T. H. Trinh, Y. Wu, Q. V. Le, H. He, and T. Luong. Solving olympiad geometry without human demonstrations. *Nat.*, 625(7995):476-482, 2024. doi: 10.1038/S41586-023-06747-5. URL <https://doi.org/10.1038/s41586-023-06747-5>.

A. Prompt Templates

A. 提示模板

A.1. Proof Generation Prompt

A.1. 生成证明的提示

Your task is to solve a given problem. The problem may ask you to
你的任务是解决一个给定的问题。该问题可能会要求你

↪ prove a statement, or ask for an answer. If finding an
↪ answer is required, you should come up with the answer, and
↪ your final solution should also be a rigorous proof of that
↪ answer being valid.

↪ 证明一个陈述，或提出一个问题。如果需要找到一个
↪ 答案，你应该想出这个答案，且
↪ 你的最终解决方案也应该是该
↪ 答案有效的严谨证明。

Your final solution to the problem should be exceptionally
你的最终解决方案应该非常出色

↪ comprehensive and easy-to-follow, which will be rated
↪ according to the following evaluation instruction:

↪ 全面且易于遵循，将根据以下评估说明进行评分
↪ 评估说明如下：

“txt

“txt

Here is the instruction to evaluate the quality of a solution to
以下是评估解决方案质量的指导方针

↪ a problem. The problem may ask for a proof of statement, or
↪ ask for an answer. If finding an answer is required, the
↪ solution should present the answer, and it should also be a
↪ rigorous proof of that answer being valid.

↪ 一个问题。这个问题可能要求证明某个陈述，或
↪ 要求给出一个答案。如果需要找到答案，那么
↪ 解决方案应呈现答案，并且还应给出一个
↪ 严谨的证明，以证明该答案的正确性。

Please evaluate the solution and score it according to the
请评估该解决方案，并根据以下标准进行评分

↪ following criteria:

↪ 以下标准：

- If the solution is completely correct, with all steps executed
↪ properly and clearly demonstrated, then the score is 1
- If the solution is generally correct, but with some details
↪ omitted or minor errors, then the score is 0.5

- If the solution does not actually address the required problem,
 ↪ contains fatal errors, or has severe omissions, then the score is 0
- 如果解决方案完全正确，所有步骤都正确且清晰地展示出来，那么得分是 1
- 如果解决方案基本正确，但有些细节
 ↪ 被省略或有小错误，那么得分是 0.5
- 如果解决方案实际上没有解决所需的问题，
 ↪ 存在致命错误或有严重遗漏，那么得分是 0

Additionally, referencing anything from any paper does not save

此外，引用任何论文中的内容并不能挽救

- ↪ the need to prove the reference. It's okay IF AND ONLY IF
- ↪ the solution also presents a valid proof of the reference
- ↪ argument(s); otherwise, if the solution omits the proof or
- ↪ if the proof provided is not completely correct, the
- ↪ solution should be scored according to the criteria above,
- ↪ and definitely not with a score of 1

↪ 需要证明参考文献。这在以下情况下是可以接受的：

- ↪ 如果解决方案还提供了参考文献的正确证明
- ↪ 论点；否则，如果解决方案省略了证明或
- ↪ 如果提供的证明不完全正确，那么
- ↪ 解决方案应按照上述标准评分，
- ↪ 绝对不能给 1 分

(

(

In fact, you already have the ability to rate your solution

事实上，你已经具备对解决方案进行评分的能力

- ↪ yourself, so you are expected to reason carefully about how to solve a given problem, evaluate your method according to
- ↪ the instruction, and refine your solution by fixing issues
- ↪ identified until you can make no further progress.

- ↪ 你自己，因此你被期望仔细思考如何解决给定的问题，根据
- ↪ 指令评估你的方法，并通过修复识别出的问题
- ↪ 不断完善你的解决方案，直到你无法进一步取得进展。

In your final response, you should present a detailed solution to

在你的最终回复中，你应该呈现一个详细的解决方案来

the problem followed by your evaluation of that solution.

问题及其解决方案的评估。

- To give a good final response, you should try your best to
- 为了给出一个良好的最终回复，你应该尽你最大的努力去

- ↪ locate potential issues in your own (partial) solution
- ↪ according to the evaluation instruction above, and fix them
- ↪ as many as you can.

- ↪ 在自己的（部分）解决方案中定位潜在问题
- ↪ 根据上面的评估说明，修复这些问题
- ↪ 尽你所能修复尽可能多的问题。

- A good final response should just faithfully present your
- 一个好的最终回复应该只是忠实地呈现你的

↪ progress, including the best solution you can give, as well
 ↪ as a faithful evaluation of that solution.

↪ 进展，包括你能提供的最佳解决方案，以及对该解决方案的忠实评估。

- Only when you fail to locate any issues in your solution should ↪ you score it with 1.
- 只有当你在你的解决方案中找不到任何问题时，才应将分数定为 1。
- If you do notice some issues in your solution but fail to
- 如果您在解决方案中注意到一些问题但未能

↪ resolve them with your best efforts, it's totally ok to

↪ faithfully present the issues in your final response.

↪ 用你们最大的努力来解决它们，完全没问题

↪ 在最终回答中忠实呈现问题。

- The worst final response would provide a wrong solution but lie
- 最糟糕的最终回应是提供一个错误的解决方案却撒谎

↪ that it's correct or claim that it's correct without

↪ careful error checking. A better version should faithfully

↪ identify errors in the solution. Remember! You CAN'T cheat!

↪ If you cheat, we will know, and you will be penalized!

↪ 说它是正确的或声称它是正确的，而没有

↪ 进行仔细的错误检查。一个更好的版本应该能够

↪ 准确识别解决方案中的错误。记住！你不能作弊！

↪ 如果你作弊，我们会知道，你将会被处罚！

Your final response should be in the following format:

你的最终回复应采用以下格式：

Solution // Your final solution should start with this exact

解决方案 // 你的最终解决方案应以以下内容开头

↪ same markdown title

↪ 相同的 markdown 标题

... // Your final solution to the problem here. You should try ↪ your best to optimize the quality of your solution according ↪ to the evaluation instruction above before finalizing it ↪ here.

... // 在此处写出你对问题的最终解决方案。你应该尽量 ↪ 努力根据上面的评估指示优化你解决方案的质量，然后再在此处最终确定它 ↪。

Self Evaluation // Your evaluation of your own solution above → should start with this exact same markdown title

自我评价 // 你对上面的解决方案的自我评估应以这个确切的 markdown 标题开始 →

Here is my evaluation of the solution: // Your analysis should → start with this exact same phrase

以下是对我解决方案的评估：// 你的分析应该 → 以这句话开头

... // Your evaluation here. You are required to present in → detail the key steps of the solution or the steps for which → you had doubts regarding their correctness, and explicitly → analyze whether each step is accurate: for correct steps, → explain why you initially doubted their correctness and why → they are indeed correct; for erroneous steps, explain the → reason for the error and the impact of that error on the → solution. You should analyze your solution faithfully. E.g. → if there are issues in your final solution, you should → point it out.

... // 你的评估在此。你需要在 $\rightarrow$ 中详细说明解决方案的关键步骤或你对 $\rightarrow$ 步骤正确性存有疑问的步骤，并明确 $\rightarrow$ 分析每一步是否正确：对于正确的步骤， $\rightarrow$ 解释你最初对其正确性产生怀疑的原因以及为什么 $\rightarrow$ 它们确实是正确的；对于错误的步骤，解释 $\rightarrow$ 错误的原因以及该错误对 $\rightarrow$ 解决方案的影响。你应该忠实分析你的解决方案。例如 $\rightarrow$ ，如果你的最终解决方案存在问题，你应当 $\rightarrow$ 指出。

Based on my evaluation, the final overall score should be: $\{\dots\}$ // where ... should be the final overall score (0, $\rightarrow$ 0.5, or 1, and nothing else) based on the evaluation $\rightarrow$ instruction above. You should reach this score ONLY AFTER $\rightarrow$ careful RE-examination of your own solution above

根据我的评估，最终的总体得分应该是： $\{\dots\}$ // 其中 ... 应该是基于上述评估 $\rightarrow$ 指令得出的最终总体得分 (0, $\rightarrow$ 0.5, 或 1, 其他内容不可用)。你应在仔细重新审视了上述自己的解决方案之后才得出这个分数 $\rightarrow$

Here is your task input:

以下是你的任务输入：

Problem {question}

问题 {question}

A.2. Proof Verification Prompt

A.2. 证明验证提示

Instruction

指令

Your task is to evaluate the quality of a solution to a problem. $\rightarrow$ The problem may ask for a proof of statement, or ask for an

你的任务是评估解决问题的方案的质量。 $\rightarrow$ 问题可能要求证明某个陈述，或者要求一个

$\hookrightarrow$ answer. If finding an answer is required, the solution

$\hookrightarrow$ should present the answer, and it should also be a rigorous

$\hookrightarrow$ proof of that answer being valid.

$\hookrightarrow$ 答案。如果需要找到答案，解决方案

$\hookrightarrow$ 应该呈现答案，并且还应是该答案有效的严谨

$\hookrightarrow$ 证明。

Please evaluate the solution and score it according to the following criteria:

请根据以下标准评估解决方案并进行评分：

- If the solution is completely correct, with all steps executed $\hookrightarrow$ properly and clearly demonstrated, then the score is 1
- 如果解决方案完全正确，所有步骤都 $\hookrightarrow$ 正确且清晰地展示出来，那么得分是 1
- If the solution is generally correct, but with some details $\leftrightarrow$ omitted or minor errors, then the score is 0.5
- 如果解决方案总体正确，但有一些细节 $\leftrightarrow$ 被省略或有小错误，则得分为 0.5
- If the solution does not actually address the required problem, $\hookrightarrow$ contains fatal errors, or has severe omissions, then the $\hookrightarrow$ score is 0
- 如果解决方案实际上没有解决所需的问题， $\hookrightarrow$ 存在致命错误，或者有严重的遗漏，那么 $\hookrightarrow$ 得分是 0
- Additionally, referencing anything from any paper does not save $\hookrightarrow$ the need to prove the reference. It's okay IF AND ONLY IF $\hookrightarrow$ the solution also presents a valid proof of the reference $\hookrightarrow$ argument(s); otherwise, if the solution omits the proof or $\hookrightarrow$ if the proof provided is not completely correct, the $\hookrightarrow$ solution should be scored according to the criteria above, $\hookrightarrow$ and definitely not with a score of 1

- 此外，引用任何论文中的内容并不能免除 $\hookrightarrow$ 需要证明引用的义务。只有在 $\hookrightarrow$ 解决方案也提供了参考 $\hookrightarrow$ 论点的有效证明时，这种情况才是可以接受的；否则，如果解决方案省略了证明或 $\hookrightarrow$ 提供的证明不完全正确，那么 $\hookrightarrow$ 解决方案应按照上述标准评分， $\hookrightarrow$ 而且绝对不能给 1 分。

Please carefully reason out and analyze the quality of the solution below, and in your final response present a detailed evaluation of the solution's quality followed by your score. Therefore, your response should be in the following format:

请仔细分析并评估以下解决方案的质量，最后在您的回复中呈现对解决方案质量的详细评估，然后给出您的评分。因此，您的回复应采用以下格式：

Here is my evaluation of the solution:

... // Your evaluation here. You are required to present in $\hookrightarrow$ detail the key steps of the solution or the steps for which $\hookrightarrow$ you had doubts regarding their correctness, and explicitly $\hookrightarrow$ analyze whether each step is accurate: for correct steps, $\hookrightarrow$ explain why you initially doubted their correctness and why $\hookrightarrow$ they are indeed correct; for erroneous steps, explain the $\hookrightarrow$ reason for the error and the impact of that error on the $\hookrightarrow$ solution.

以下是对我解决方案的评估：

... // 你的评估内容。你需要在 $\hookrightarrow$ 中详细说明解决方案的关键步骤，或者说明你对哪些步骤 $\hookrightarrow$ 存在对其正确性的疑问，并明确 $\hookrightarrow$ 分析每一步是否正确：对于正确的步骤， $\hookrightarrow$ 解释你最初对其正确性产生怀疑的原因，以及 $\hookrightarrow$ 为什么它们实际上是正确的；对于错误的步骤，解释 $\hookrightarrow$ 错误的原因以及该错误对 $\hookrightarrow$ 解决方案的影响。

Based on my evaluation, the final overall score should be: $\boxed{\{...\}}$ // where ... should be the final overall score (0, $\rightarrow$ 0.5, or 1, and nothing else) based on the above criteria

根据我的评估，最终的总体评分应为： $\boxed{\{...\}}$ // 其中 ... 应该是基于上述标准的最终总体评分（0， $\rightarrow$ 0.5，或 1，其他内容不可用）

Here is your task input:

以下是你的任务输入：## Problem

问题

{question}
{question} ## Solution

解决方案

{proof}
{proof}

A.3. Meta-Verification Prompt

A.3. 元验证提示

You are given a “problem”, “solution”, and “solution evaluation”, $\hookrightarrow$ and you need to assess the whether this “solution

你被提供了一个“问题”、“解决方案”和“解决方案评估”， $\hookrightarrow$ ，你需要评估这个“解决方案”是否正确。

$\hookrightarrow$ evaluation” is reasonable.

$\hookrightarrow$ 评估”是合理的。

First, “solution evaluation” is generated to evaluate the quality $\hookrightarrow$ of the “solution”, by prompting a verifier with the rules $\hookrightarrow$ below (these are not your rules):

首先，生成“解决方案评估”以评估“解决方案”的质量 $\hookrightarrow$ ，通过提示验证者使用以下规则 $\hookrightarrow$ （这些规则不是您的规则）：

Please evaluate the solution and score it according to the following criteria:

请根据以下标准评估解决方案并进行评分：

- If the solution is completely correct, with all steps executed properly and clearly demonstrated, then the score is 1
- 如果解决方案完全正确，所有步骤都执行得当且清晰展示，那么得分是 1
- If the solution is generally correct, but with some details $\hookrightarrow$ omitted or minor errors, then the score is 0.5
- 如果解决方案总体正确，但有一些细节 $\hookrightarrow$ 被省略或有小错误，则得分为 0.5
- If the solution does not actually address the required problem, $\hookrightarrow$ contains fatal errors, or has severe omissions, then the $\hookrightarrow$ score is 0
- 如果解决方案实际上没有解决所需的问题， $\hookrightarrow$ 存在致命错误，或者有严重遗漏，则 $\hookrightarrow$ 得分是 0

Additionally, referencing anything from any paper does not save $\hookrightarrow$ the need to prove the reference. It's okay IF AND ONLY IF $\hookrightarrow$ the solution also presents a valid proof of the reference $\hookrightarrow$ argument(s); otherwise, if the solution omits the proof or $\hookrightarrow$ if the proof provided is not completely correct, the $\hookrightarrow$ solution should be scored according to the criteria above, $\hookrightarrow$ and definitely not with a score of 1

此外，引用任何论文中的内容并不能免除 $\hookrightarrow$ 证明引用的需要。只有在 $\hookrightarrow$ 解决方案也提供了引用 $\hookrightarrow$ 论点的有效证明时，这种情况才是可以接受的；否则，如果解决方案省略了证明或 $\hookrightarrow$ 提供的证明不完全正确，那么 $\hookrightarrow$ 解决方案应根据上述标准进行评分， $\hookrightarrow$ 绝对不能给出 1 分。

Next, I will introduce the rules for you to analyze the quality $\hookrightarrow$ of the “solution evaluation”:

接下来，我将向您介绍分析“解决方案评估”中 $\hookrightarrow$ 质量的规则：

1. Your task is to analyze the “solution evaluation”. You do not need to solve the “problem”, nor do you need to strictly assess whether the “solution” is accurate. Your only task is to strictly follow the rules below to evaluate whether the “solution evaluation” is reasonable.
2. 你的任务是分析“解决方案评估”。你不需要解决“问题”，也不需要严格评估“解决方案”是否准确。你的唯一任务是严格按照以下规则来评估“解决方案评估”是否合理。
3. You need to analyze the content of the “solution evaluation” $\rightarrow$ from three aspects:
4. 你需要从三个方面分析“解决方案评估” $\rightarrow$ 的内容：

Step Restatement: In the “solution evaluation”, certain behaviors $\hookrightarrow$ of the “solution” may be restated. You need to return to $\hookrightarrow$ the original text of the “solution” and check whether the “solution” actually has these behaviors mentioned in the “solution evaluation”.

步骤重述：在“解决方案评估”中，某些行为 $\hookrightarrow$ 可能被重新表述。你需要返回到 $\hookrightarrow$ 查看“解决方案”的原文，检查“解决方案”是否实际上具有“解决方案评估”中提到的这些行为。

Defect Analysis: “solution evaluation” may point out errors or $\hookrightarrow$ defects in the “solution”. You need to carefully analyze $\hookrightarrow$ whether the mentioned errors and defects are indeed valid.

缺陷分析：“解决方案评估”可能指出“解决方案”中的错误或 $\hookrightarrow$ 缺陷。您需要仔细分析 $\hookrightarrow$ 所提到的错误和缺陷是否确实有效。

Expression Analysis: Whether the “solution evaluation”’s $\hookrightarrow$ expressions are accurate.

表达分析：“解决方案评估”的 $\hookrightarrow$ 表达是否准确。

Score Analysis: Whether the final score given by the “solution $\hookrightarrow$ evaluation” matches the defects it found. You need to $\hookrightarrow$ analyze according to the scoring rules given above.

评分分析：评分是否与“解决方案 $\hookrightarrow$ 评估”所发现的缺陷相匹配。你需要 $\hookrightarrow$ 根据上述评分规则进行分析。

3. The most important part is **defect analysis**: In this part, $\hookrightarrow$ your core task is to check whether the errors or defects of $\hookrightarrow$ the “solution” pointed out in the “solution evaluation” are $\hookrightarrow$ reasonable. In other words, any positive components about $\hookrightarrow$ the “solution” in the “solution evaluation”, regardless of $\hookrightarrow$ whether they are reasonable, are not within your evaluation $\hookrightarrow$ scope.
4. 最重要的一部分是 **缺陷分析**：在这一部分中， $\hookrightarrow$ 你的核心任务是检查 $\hookrightarrow$ 的错误或缺陷是否与“解决方案”所指出的“解决方案评估”中的“解决方案” $\hookrightarrow$ 合理。换句话说，无论 $\hookrightarrow$ “解决方案”中的任何正面成分是否合理， $\hookrightarrow$ 都不在你的评估 $\hookrightarrow$ 范围之内。
 - For example: If the “solution evaluation” says that a certain $\hookrightarrow$ conclusion in the “solution” is correct, but actually this $\hookrightarrow$ conclusion is incorrect, then you do not need to care about $\hookrightarrow$ this point. All parts that the “solution evaluation” $\hookrightarrow$ considers correct do not belong to your evaluation scope.
 - Specifically: If the “solution evaluation” believes that the “solution” is completely accurate and has not found any errors or defects, then regardless of whether the “solution itself is actually accurate, even if there are obvious errors, you should still consider its analysis of errors to be reasonable.
 - 例如：如果“解决方案评估”指出“解决方案”中的某个 $\hookrightarrow$ 结论是正确的，但实际上这个 $\hookrightarrow$ 结论是错误的，那么你不需关注 $\hookrightarrow$ 这个问题。所有“解决方案评估” $\hookrightarrow$ 认为正确的部分都不属于你的评估范围。
 - 具体来说：如果“解决方案评估”认为“解决方案”完全准确，没有发现任何错误或缺陷，那么无论“解决方案本身是否实际上准确，即使存在明显的错误，你也应该认为其对错误的分析是合理的。

Importantly, for defects found by the “solution evaluation”, you need to analyze two points simultaneously:

重要的是，对于通过“解决方案评估”发现的缺陷，您需要同时分析两个要点：

- whether this defect actually exists
- whether the “solution evaluation”’s analysis of this defect is $\leftrightarrow$ accurate
- 这个缺陷是否实际上存在
- “解决方案评估”对这个缺陷的分析是否 $\leftrightarrow$ 准确

These two aspects constitute the analysis of defects.

这两个方面构成了缺陷分析。

4. About **expression analysis**, if there are certain expression errors in the “solution evaluation”, even minor errors in details, you need to identify them. However, please note that identifying incorrect steps in the “solution” as correct steps does not constitute an **expression error**.
5. 关于 **表达分析**，如果在“解决方案评估”中存在某些表达错误，即使是细节中的小错误，你也需要识别出来。然而，请注意，将“解决方案”中的错误步骤误认为正确步骤，并不构成 **表达错误**。

In practice, expression errors include but are not limited to:

实际上，表达错误包括但不限于：

- If the “solution evaluation” identifies some reasoning step(s) $\hookrightarrow$ in the “solution” as incorrect, then it cannot further $\hookrightarrow$ indicate that subsequent conclusion(s) depending on those $\hookrightarrow$ reasoning step(s) are wrong, but can only indicate that $\hookrightarrow$ subsequent conclusion(s) are “not rigorously demonstrated.”
- 如果“解决方案评估”识别出“解决方案”中的某些推理步骤 $\hookrightarrow$ 是错误的，那么它不能进一步 $\hookrightarrow$ 指出依赖于这些 $\hookrightarrow$ 推理步骤的后续结论是错误的，但只能指出这些 $\hookrightarrow$ 后续结论是“未严格证明的。”

- Typos and calculation errors made by “solution evaluation”
- Inaccurate restatement of content from “solution”
- “解决方案评估”所造成的拼写错误和计算错误

- 对“解决方案”内容的不准确重述

5. Finally, you need to present your analysis of the “solution $\hookrightarrow$ evaluation” in your output and also rate its quality based on the rules below:

6. 最后，你需要在输出中展示你对“解决方案 $\hookrightarrow$ 评估”的分析，并根据以下规则对其质量进行评分：

First, if there is at least one unreasonable defect among the $\hookrightarrow$ defects found by the “solution evaluation”, then you only $\hookrightarrow$ need to do **defect analysis**:

首先，如果在“解决方案评估”找到的 $\hookrightarrow$ 缺陷中至少有一个不合理缺陷，那么你只需 $\hookrightarrow$ 进行 **缺陷分析**：

- If all defects found by the “solution evaluation” are $\hookrightarrow$ unreasonable, then you should rate it with $\backslash(0\backslash)$
- 如果“解决方案评估”中发现的所有缺陷都被认为是 $\hookrightarrow$ 不合理，那么您应该用 $\backslash(0\backslash)$ 进行评分
- If some defects found by the “solution evaluation” are $\rightarrow$ reasonable and some are unreasonable, then your rating $\rightarrow$ should be $(0.5\backslash)$ Next, if the “solution evaluation” points out no errors or $\rightarrow$ defects, or all defects found by the evaluation are $\rightarrow$ reasonable, then you should do the following things:

- Analyze whether “expression errors” exist in the “solution $\rightarrow$ evaluation” (**expression analysis) or whether “solution $\rightarrow$ evaluation” gives a wrong score according to the rules for “ $\rightarrow$ solution evaluation” (**score analysis). If yes, you $\rightarrow$ should rate the “solution evaluation” with $\backslash(0.5\backslash)$; if no, $\rightarrow$ your rating should be $(1\backslash)$ Your output should follow the format below:

Here is my analysis of the “solution evaluation”: ... // Your analysis here..

Based on my analysis, I will rate the “solution evaluation” as: $\boxed{\{...\}}$ // where ... should be a numerical rating of the “ $\rightarrow$ solution evaluation” (0, 0.5, or 1, and nothing else) based $\rightarrow$ on the criteria above.

—

Here is your task input:

- 如果通过“解决方案评估”发现的一些缺陷是 $\rightarrow$ 合理的，而另一些是不合理的，那么你的评分 $\rightarrow$ 应该是 $(0.5\backslash)$ 接下来，如果“解决方案评估”指出没有错误或 $\rightarrow$ 缺陷，或者评估发现的所有缺陷都是 $\rightarrow$ 合理的，那么你应该执行以下操作：

- 分析“解决方案 $\rightarrow$ 评估”中是否存在“表达错误”（**表达分析**），或者“解决方案 $\rightarrow$ 评估”是否根据“ $\rightarrow$ 解决方案评估”的规则给出了错误的分数（**评分分析**）。如果存在，你 $\rightarrow$ 应该对“解决方案评估”进行 $\backslash(0.5\backslash)$ 评分；如果不存在， $\rightarrow$ 你的评分应该是 $(1\backslash)$ 你的输出应遵循以下格式：

这里是对我对“解决方案评估”的分析：... // 你的分析在此。

基于我的分析，我将对“解决方案评估”进行如下评分： $\boxed{\{...\}}$ // 其中...应该是基于上述标准对“ $\rightarrow$ 解决方案评估”进行的数值评分（0，0.5，或 1，其他内容除外）。

—

这里是你的任务输入：## Problem {question}

问题 {question}

Solution {proof}

解决方案 {proof}

Solution Evaluation {proof analysis}

解决方案评估 {证明分析}

A.4. Proof Refinement Prompt

A.4. 证明细化提示

{proof_generation_prompt}

{proof_generation_prompt} ## Candidate Solution(s) to Refine

候选解决方案以优化

Here are some solution sample(s) along with their correctness → evaluation(s). You should provide a better solution by → solving issues mentioned in the evaluation(s), or by re- → using promising ideas mentioned in the solution sample(s), → or by doing both.

{proof}

{proof analyses}

以下是一些解决方案示例及其正确性 → 评估。您应该通过 → 解决评估中提到的问题，或通过重新 → 使用解决方案示例中提到的有希望的想法 →，或者两者都这样做来提供更好的解决方案。

{proof}

{proof analyses} ## Final Instruction

最终指令

Your final response should follow the format above, including a → ‘## Solution’ section followed by a ‘# Self Evaluation’

解决方案

→

↪ section

↪ 部分